

# Accelerating an Exact Unknot Recognizer

Compiled cobordisms, low-fill cancellation, scalar obstructions,  
and connected-sum products

Implementation and reproducibility report

Prepared for Vladimir Reshetnikov

18 September 2026

## Abstract

We replace the implementation in the supplied `Knots.zip/fast/` with a standard-library Python implementation retaining its complete Khovanov-based decision procedure. The changes are executable, tested, and measured against an unchanged copy of the original source. They include compiled bitset-valued cobordism maps, sparse low-fill pivot selection, a faster construction of the same crossing orders, exact one-sided modular Alexander and Jones tests, and visible connected-sum factorization.

The original 36-crossing five-strand benchmark does not finish a 60-second budget on the test host. The new raw backend computes its reduced  $\mathbb{F}_2$  rank, 2949, in 0.897 seconds (median of three fresh processes). The original explicit surface algebra, with only its pivot scheduler changed, independently agrees in every homological degree. Complete recognition of the supplied Conway and Kinoshita–Terasaka diagrams improves by factors 26.9 and 29.3, respectively. On visible connected sums of a fixed Alexander-trivial knot, the original pipeline’s explicit exponential-rank expansion is replaced by polynomial-size exact arithmetic. Fifty Conway summands, with reduced rank  $33^{50}$ , are evaluated in 0.163 seconds.

These results do *not* establish a general quasi-polynomial bound. We identify why a boundary-width argument does not provide one, distinguish the supplied Lackenby hierarchy material from this implementation, and retain an honest  $2^{O(n)}$  general bound. All 252 benchmark samples, source provenance, 34 test methods, and reproduction scripts accompany the report.

# Contents

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Scope, baseline, and mathematical contract</b>                    | <b>3</b>  |
| <b>2</b>  | <b>The quasi-polynomial target: what transfers and what does not</b> | <b>3</b>  |
| 2.1       | Reading the supplied Lackenby material quantitatively . . . . .      | 3         |
| 2.2       | A concrete attempted transfer to the supplied scanner . . . . .      | 4         |
| <b>3</b>  | <b>Compiling the dotted-cobordism algebra</b>                        | <b>5</b>  |
| 3.1       | Separate topology from coefficients . . . . .                        | 5         |
| 3.2       | The component evaluation formula . . . . .                           | 5         |
| 3.3       | Every relevant unit is an involution . . . . .                       | 6         |
| 3.4       | Memory tradeoffs . . . . .                                           | 7         |
| <b>4</b>  | <b>Sparse cancellation and order construction</b>                    | <b>7</b>  |
| 4.1       | Exact cancellation by a Schur complement . . . . .                   | 7         |
| 4.2       | Lazy Markowitz-style priorities . . . . .                            | 7         |
| 4.3       | The same greedy order in less time . . . . .                         | 8         |
| <b>5</b>  | <b>Exact one-sided obstructions before the full complex</b>          | <b>8</b>  |
| 5.1       | Numeric Alexander minors . . . . .                                   | 8         |
| 5.2       | Scalar Jones contraction . . . . .                                   | 9         |
| 5.3       | Orientation and witness details . . . . .                            | 10        |
| <b>6</b>  | <b>Visible connected sums without generator expansion</b>            | <b>10</b> |
| 6.1       | Finding and validating a two-edge split . . . . .                    | 10        |
| 6.2       | Rank products and degree convolution . . . . .                       | 11        |
| 6.3       | An actual family-level asymptotic improvement . . . . .              | 11        |
| <b>7</b>  | <b>Complexity and resource semantics</b>                             | <b>12</b> |
| 7.1       | What is, and is not, bounded . . . . .                               | 12        |
| 7.2       | Practical ceilings . . . . .                                         | 12        |
| <b>8</b>  | <b>Validation and independent checks</b>                             | <b>13</b> |
| 8.1       | The test corpus . . . . .                                            | 13        |
| 8.2       | The difficult 36-crossing result . . . . .                           | 13        |
| <b>9</b>  | <b>Same-host performance measurements</b>                            | <b>14</b> |
| 9.1       | Protocol . . . . .                                                   | 14        |
| 9.2       | Raw backend: no preprocessing and no factorization . . . . .         | 14        |
| 9.3       | Complete recognition pipeline . . . . .                              | 15        |
| 9.4       | Rank products on visible connected sums . . . . .                    | 16        |
| 9.5       | Ablations and crossing-order cost . . . . .                          | 16        |
| <b>10</b> | <b>Using, auditing, and extending the implementation</b>             | <b>17</b> |
| 10.1      | Commands and API . . . . .                                           | 17        |
| 10.2      | Source organization . . . . .                                        | 17        |
| 10.3      | Reproduction and remaining technical work . . . . .                  | 18        |
| <b>A</b>  | <b>The hard input and its complete recorded rank distribution</b>    | <b>18</b> |
| <b>B</b>  | <b>Provenance and interpretation of evidence</b>                     | <b>19</b> |

# 1 Scope, baseline, and mathematical contract

The input is a classical, one-component knot diagram with  $n$  crossings. Edge labels are normalized to  $0, \dots, 2n - 1$ . A planar-diagram (PD) row  $(a, b, c, d)$  is counterclockwise, with under-strand  $a-c$  and over-strand  $b-d$ . Each edge label appears twice. The parser checks the strand permutation and the spherical rotation system, rather than accepting an arbitrary abstract four-valent graph as a classical diagram. The empty PD means one crossing-free circle, not the empty link.

The baseline is the actual Python package in the uploaded `fast/` directory. It is retained byte-for-byte in `baseline/fastunknot/`. The archive’s pre-existing timings were *not* reused as new measurements. Its benchmark words were reused as input data, including the word for which the archived run exhausted 600 seconds on a different platform. All ratios reported here use new measurements on the same host and interpreter.

The baseline first applies decreasing Reidemeister I/II moves, then a descending diagram test, then an exact Alexander polynomial filter. It next constructs a Khovanov complex over  $\mathbb{F}_2$  by scanning crossings, delooping closed circles, and cancelling invertible differential entries. The algebraic structure follows the local-complex approach of Bar-Natan [3]. The new implementation retains this route to a complete answer; its extra filters never substitute “a familiar-looking invariant” for a proof of triviality.

Write

$$r_2(K) = \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2).$$

The required detection theorem is  $r_2(K) = 1$  if and only if  $K$  is the unknot. Indeed, the integral universal coefficient theorem gives  $\dim_{\mathbb{Q}} \widetilde{\text{Kh}}(K; \mathbb{Q}) \leq r_2(K)$ ; reduced rational homology is nonzero, and the rank-one detection theorem of Kronheimer–Mrowka applies [4]. Conversely the unknot has reduced dimension one directly. The characteristic-two reduced/unreduced splitting preserves homological degree and doubles total dimension; see Shumakovitch, Corollary 3.2.C [5]. Thus the scanner’s unreduced rank is divided by two. Quantum gradings are forgotten throughout this implementation.

**Proposition 1.1** (Decision contract). *Assuming the standard Khovanov invariance and detection theorems, every completed arithmetic stage in the new implementation has the following meaning:*

1. *Decreasing moves ending at a crossing-free circle, or a descending presentation, certify **UNKNOT**.*
2. *A nonunit Alexander polynomial, a valid modular Alexander obstruction, or a Jones evaluation different from the unknot certifies **KNOTTED**.*
3. *Otherwise the exact reduced Khovanov rank determines the answer.*

*Optional filter limits do not create a verdict. Exhaustion of a global resource budget returns **UNKNOWN**.*

The public rank output is total unreduced and reduced  $\mathbb{F}_2$  dimension, together with unreduced dimensions in unnormalized cube degree. It is not integer Khovanov homology, a quantum-graded table, or a grading-normalized invariant. Total rank is unaffected by grading shifts; cube-degree distributions can shift when a diagram is changed by Reidemeister moves.

## 2 The quasi-polynomial target: what transfers and what does not

### 2.1 Reading the supplied Lackenby material quantitatively

The supplied talk announces quasi-polynomial unknot recognition. Its final flowchart uses layered handle structures, hierarchical multi-surfaces, Cheeger-region operations, and weak reduction of generalized Heegaard splittings [1]. These are not operations on the boundary matchings of a Khovanov scan.

The supplied July 2026 paper presents a hierarchy-based incompressibility algorithm. Proposition 9.1 bounds iterations by  $L(g+1)^L$ , where  $L$  bounds hierarchy length and  $g$  bounds surface pattern-complexity. Section 9 separates this count from the cost of executing its steps. Importantly, the subsequent refinement *does* bound hierarchy length: Proposition 10.2 gives  $L \leq 4c_q(\mathcal{H})$ , with  $c_q(\mathcal{H}) = \sum_H \max\{I(H), 0\}^2$  [2]. It would be inaccurate to read Section 9 in isolation and say that the entire paper leaves  $L$  without a quantitative bound.

Here is the relevant arithmetic of such a bound. If one had, for an executable hierarchy representation,

$$g \leq n^a, \quad L \leq b \log n,$$

then

$$L(g+1)^L \leq O(\log n) \exp(O((\log n)^2)) = n^{O(\log n)}.$$

One would still need to control the bit lengths of the represented surfaces and the cost of every cut, compression, connectivity query, and restoration step. A count of abstract iterations alone does not control arithmetic on exponentially expanded surfaces. Conversely, even a bound  $L = O(n)$  is not a logarithmic bound: combined with  $g = n^{O(1)}$ , the same estimate only gives  $n^{O(n)}$ . These are elementary deductions from the potential, not a claim that this is the only possible analysis of the hierarchy method.

## 2.2 A concrete attempted transfer to the supplied scanner

The natural proposed transfer is to regard a narrow crossing boundary as a small interface and to bound the state at that interface by its matching type. One might then hope that balanced splitting, logarithmic recursion depth, or a small-width ordering yields a quasi-polynomial scanner. The following obstruction shows precisely why that argument fails for the implemented explicit complex.

**Proposition 2.1** (Width does not bound categorical multiplicity). *There is a family of  $O(k)$ -crossing knot diagrams with uniformly bounded-width layouts whose reduced  $\mathbb{F}_2$  Khovanov ranks are  $3^k$ . Consequently, the explicit final-generator representation used by the original scanner admits no general bound  $\text{poly}(n)f(w)$  depending only on the crossing-boundary width  $w$ .*

*Proof.* Join  $k$  fixed trefoil diagrams in a chain, using two-edge connected-sum interfaces, and process the blocks successively. At any point only a constant number of edges of the current fixed-size block, plus the joining interface, are exposed. The width is therefore bounded independently of  $k$ . Reduced homology of a connected sum is a tensor product over the coefficient field, so its dimension is  $3^k$ ; the chain-level explanation is given in Section 6. The final boundary is empty, yet the original scanner retains  $2 \cdot 3^k$  unreduced generators. It must represent them before counting them. Thus even width zero at the final stage carries exponentially large multiplicity.  $\square$

At a boundary of  $w$  labeled ends there are at most

$$(w-1)!! = \frac{w!}{2^{w/2}(w/2)!} = 2^{O(w \log w)}$$

perfect matching types. This count does not include how many copies of a type occur in each homological degree. It also does not automatically improve to a Catalan number: that requires a compatible disk-boundary ordering and noncrossing matchings relative to that ordering, neither of which is stored as a general invariant of the scanner's exposed interface.

A balanced recursion tree on crossings controls tree depth, not the sizes of its intermediate complexes. Likewise, a good graph layout alone does not remove the above homological output. The

new connected-sum representation repairs one specific multiplicity explosion by storing products and coefficient polynomials. It is not a replacement for the handle/hierarchy invariants in the talk. No hierarchy engine, compressed normal-surface implementation, or quasi-polynomial certificate search is hidden behind the new API.

### 3 Compiling the dotted-cobordism algebra

#### 3.1 Separate topology from coefficients

An object in the scanned complex consists of a matching  $a$  on the current boundary and a homological degree. Glue a source matching  $a$  to a target matching  $b$  along their ends. Suppose the result has circles  $C_0, \dots, C_{q-1}$ . A basis for the simplified dotted morphisms consists of a disk on each circle with either zero or one dot. Encode its dot set by

$$m = \sum_{i \in S} 2^i, \quad 0 \leq m < 2^q.$$

A general  $\mathbb{F}_2$  morphism is encoded as the Python integer

$$\mathbf{f} = \sum_{m \in E} 2^m.$$

The outer exponent is deliberate:  $m$  is a dot mask, whereas  $\mathbf{f}$  is a coefficient bitset indexed by dot masks. Confusing those two integers would produce an incorrect implementation.

For instance, with two basis circles, the morphism  $1 + x_0 + x_0 x_1$  has dot-mask indices 0, 1, 3 and coefficient integer  $1 + 2 + 8 = 11$ . Addition is integer XOR. Zero is 0. For equal matchings, the identity is the undotted basis morphism, encoded by 1, not by a full set of low bits.

The old implementation repeatedly assembled connected surfaces for individual coefficient compositions and individual transferred differentials. The new **Algebra** compiles topology once for each matching combination within a crossing stage. A plan records, for every connected component, input-circle masks, output-circle masks, dot offsets from caps, and the surface genus. Subsequent applications use only masks, population counts, and XOR until a new matching combination is encountered.

#### 3.2 The component evaluation formula

The underlying Frobenius algebra is

$$A = \mathbb{F}_2[x]/(x^2), \quad \epsilon(1) = 0, \quad \epsilon(x) = 1,$$

with multiplication determined by  $x^2 = 0$  and comultiplication

$$\Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x.$$

These are also the operations in the independent dense-cube test oracle. The construction is the ordinary Khovanov specialization [6]; the implementation-level formulas below are obtained by evaluating these operations and the surface plans inherited from the supplied explicit algebra.

Let a connected genus-zero component have  $b$  outgoing boundary circles and  $d$  dots in total, including input dots and cap/cup contributions. It evaluates to

$$E(b, d) = \begin{cases} 0, & d \geq 2, \\ 1, & b = 0, d = 1, \\ 0, & b = 0, d = 0, \\ x_1 \cdots x_b, & b > 0, d = 1, \\ \sum_{j=1}^b \prod_{i \neq j} x_i, & b > 0, d = 0. \end{cases} \quad (1)$$

To see the last two cases, iterate  $\Delta$  on  $x$  or 1. The statement follows by induction on  $b$ : splitting a dotted output produces two dotted outputs, whereas splitting the unique undotted output produces the two possible new locations for that undotted output. Closing the last output applies  $\epsilon$ . Two dots give a factor  $x^2$  and hence zero.

A handle also gives zero in this specialization, because

$$\mu\Delta(1) = 2x = 0, \quad \mu\Delta(x) = 0.$$

Thus every positive-genus component makes the entire plan zero before any coefficient enumeration. For a union of components, multiply their output expressions. Their output-circle masks are disjoint, so multiplication here is bitwise union of the masks, not a general multiplication of two overlapping dot polynomials.

For composition, the compiler starts with the source–middle and middle–target disk bases. It glues along the middle matching. If a component contains  $v$  disks and  $e$  glued intervals, its Euler characteristic is  $\chi = v - e$ . Its outgoing boundary count is  $b$ , so

$$2g = 2 - \chi - b.$$

A negative or odd value triggers an arithmetic error rather than silently producing a result. The coefficient evaluator counts the dots in that component using masked `bit_count()` calls and applies (1).

Crossing transfer uses the same idea with the local saddle or local identity surface included in the plan. A newly closed circle is delooped into two labels. Source caps use the label as their dot count and target caps use its complement, exactly as in the explicit reference’s dual basis. This lets the scanner reuse one map for many differential entries having the same boundary matchings. It does not merge different homological generators merely because they have the same matching.

### 3.3 Every relevant unit is an involution

**Lemma 3.1.** *For a matching with  $q$  arcs, every unit of*

$$\text{End}(a) \cong \mathbb{F}_2[x_1, \dots, x_q] / (x_1^2, \dots, x_q^2)$$

*is its own inverse. In the coefficient-bitset encoding, an element is a unit exactly when its least significant bit is one.*

*Proof.* The ideal generated by the  $x_i$  is nilpotent, and the quotient by this ideal is  $\mathbb{F}_2$ . Hence a unit is precisely  $1 + \nu$  with zero constant term in  $\nu$ . In characteristic two, the square of a sum is the sum of squares. Every positive-degree square-free monomial contains some  $x_i$ , and its square therefore vanishes. Thus  $\nu^2 = 0$ , even when  $\nu$  involves many variables, and  $(1 + \nu)^2 = 1$ .  $\square$

The original inverse routine expands a finite geometric series. The new routine simply returns the same coefficient bitset. A stronger short circuit is available when composing an endomorphism with itself: its square is its constant coefficient. This is *not* a general rule for arbitrary rings, characteristics, or two different morphisms. The code checks equal source, middle, and target matchings before using it.

### 3.4 Memory tradeoffs

Integer bitsets remove Python set/frozenset allocation and perform many coefficient operations in native integer loops. They are not uniformly sparse representations. A single monomial with mask  $2^q - 1$  requires an integer with  $2^q$  bits, even though only one coefficient is nonzero. This can be worse than the old representation for very wide, sparsely supported morphisms.

The new caches are local to one scan and reset after each crossing. Scalar composition and transfer caches are each capped at 32,768 entries and cleared when full. Structural plan dictionaries live for the current stage but are not separately byte-capped. Therefore “bounded cache entries” does not mean bounded total memory. The compatibility module’s small LRU caches are bypassed by the hot path. Memory exhaustion is reported as a resource failure, not as evidence of knottedness.

## 4 Sparse cancellation and order construction

### 4.1 Exact cancellation by a Schur complement

Suppose one differential block is

$$d = \begin{pmatrix} \varphi & \delta \\ \gamma & \varepsilon \end{pmatrix}, \quad \varphi : B \longrightarrow C$$

and  $\varphi$  is invertible. Elementary changes of basis give

$$\begin{pmatrix} 1 & 0 \\ -\gamma\varphi^{-1} & 1 \end{pmatrix} d \begin{pmatrix} 1 & -\varphi^{-1}\delta \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \varphi & 0 \\ 0 & \varepsilon - \gamma\varphi^{-1}\delta \end{pmatrix}.$$

The  $B \rightarrow C$  summand is contractible. Removing it gives the homotopy-equivalent complex with updated block  $\varepsilon - \gamma\varphi^{-1}\delta$ . Over  $\mathbb{F}_2$  the minus sign is plus. This is the cancellation principle used by the original scanner; see also Bar-Natan’s Gaussian-elimination lemma [3].

The implementation stores outgoing sparse maps and incoming neighbor sets. For a pivot  $b \rightarrow c$ , it updates every pair consisting of an incoming edge to  $c$  other than  $b \rightarrow c$ , and an outgoing edge from  $b$  other than that pivot. It then removes both objects and their incidence entries. The half-composition with  $\varphi^{-1}$  is reused for all outgoing neighbors of a fixed incoming neighbor.

### 4.2 Lazy Markowitz-style priorities

The original scheduler takes unit entries from a last-in/first-out list. That choice can cause substantial fill-in. The new default prioritizes

$$\text{cost}(b, c) = (\deg^+(b) - 1)(\deg^-(c) - 1),$$

the number of candidate Schur-complement updates. It is a cheap structural proxy: cancellations may turn some candidate updates into zero, and others may hit already existing entries.

The heap stores the score and the two object IDs. Removed pivots are discarded. A popped score is recomputed; if stale, it is reinserted. Newly created unit entries are inserted immediately. Degree changes in other pivots do not trigger an eager global heap update. Accordingly this is a *lazy Markowitz-style heuristic*, not an implementation that promises the exact global minimum score at every cancellation.

Correctness is independent of the choice among valid pivots. Once the graph stops changing, stale scores settle, and every remaining queued unit is considered. A nonunit can become a unit only when its coefficient changes, at which point it is queued. Hence the scheduler does not change the final homology. The `lifo` option remains available for comparisons.

For the hard 36-crossing example, merely packing coefficients while retaining LIFO still exhausts a 10-second budget. Changing the scheduler in the original explicit algebra permits completion; combining the scheduler and compiled algebra is faster again. Thus the main improvement is not just a different spelling of the same primitive operations.

### 4.3 The same greedy order in less time

The baseline chooses a crossing sharing as many exposed edges as possible, with loop and index tie-breakers, and tries several starting crossings. Its second score component contains a term depending on the current total boundary size. That term is common to all candidates. After the shared-edge count is fixed, the effective score is simply

$$(\text{shared darts, self-loops, } -\text{crossing index}).$$

The new implementation maintains this score in a versioned heap. Processing a crossing can change only scores of crossings incident to its four edge labels; there are at most four distinct neighboring crossings. Stale entries are ignored on extraction. Thus one start costs  $O(n \log n)$  heap operations, instead of the baseline's  $\Theta(n^2)$  rescoring. The exact start set and all tie-breakers are preserved, including the original rounding behavior when the requested number of starts does not divide  $n$ .

The default tries a bounded number of starts (approximately twelve). An explicit request to try all starts has  $O(n^2 \log n)$  rather than cubic order construction. These bounds concern *order construction*, not the ensuing Khovanov computation. Tests compare the actual permutations, not merely their widths, on fifty fresh random diagrams.

## 5 Exact one-sided obstructions before the full complex

### 5.1 Numeric Alexander minors

The full polynomial filter is retained, but many inputs can be rejected without any arithmetic in  $\mathbb{Z}[t]$ . Build the same Wirtinger/Fox Alexander matrix, delete its last row and column, and evaluate this  $(n-1) \times (n-1)$  minor at  $t = -1$  and then  $t = 2$  in

$$p = 2^{31} - 1 = 2147483647.$$

Modular elimination uses a nonzero pivot and its field inverse. Updating only the nonzero tail of the pivot row often reduces work, although the dense worst case remains  $O(n^3)$  field operations.

For a knot, this diagrammatic minor agrees with the Alexander polynomial up to a unit  $\pm t^j$ . The chosen integral rows have degree at most one; hence the minor has degree at most  $n-1$  and no negative powers. For an unknot it must therefore be one of

$$\{\pm t^j : 0 \leq j \leq n-1\}.$$

At a nonzero evaluation point  $t_0$ , a determinant outside the set

$$U_n(t_0) = \{\pm t_0^j \bmod p : 0 \leq j \leq n-1\}$$

is an exact obstruction. At  $-1$  this set reduces to  $\{1, -1\}$ . At 2, testing only  $\{1, -1\}$  would be wrong: the unknown monomial unit must be respected. The implementation explicitly constructs  $U_n(2)$ .

No probabilistic inference is made when the determinant belongs to this set. A nontrivial polynomial can pass either or both tests. The later Jones stage, full polynomial stage, and complete homology stage remain available. Modular arithmetic bounds coefficient size, not the size of every later stage.

## 5.2 Scalar Jones contraction

Use the Kauffman bracket state sum, normalized consistently with the PD convention. Assign weight  $A$  to smoothing  $(0, 1)(2, 3)$  and  $A^{-1}$  to smoothing  $(0, 3)(1, 2)$ . Set

$$\delta = -A^2 - A^{-2}, \quad B(D) = \sum_{s \in \{0,1\}^n} A^{n-2|s|} \delta^{c(s)}.$$

Here every state circle contributes  $\delta$ , including the first. Thus a crossing-free circle has value  $\delta$ , rather than 1. The bracket and writhe normalization are standard; Kauffman's exposition gives their broader context [7]. With the package's oriented PD sign  $w(D)$ , the quantity

$$J(D) = \frac{B(D)}{\delta(-A^3)^{w(D)}}$$

is one on every unknot diagram.

The implementation does not enumerate all  $2^n$  states separately. At an intermediate cut, it stores one field coefficient for each boundary matching. For each crossing it extends each matching by its two smoothings. A disjoint-set calculation identifies any newly closed circles. Their powers of  $\delta$  are multiplied in immediately; equal remaining matchings are added in  $\mathbb{F}_p$ . This is a scalar dynamic program, not a complex with separate generators for every homological degree.

**Lemma 5.1** (Scalar contraction invariant). *After any initial segment of the chosen crossing order, the coefficient stored for a boundary matching is the sum of weights of all partial resolutions producing that matching, with all already closed circles evaluated by  $\delta$ .*

*Proof.* The empty partial diagram has one empty matching with coefficient one. Extending by either smoothing multiplies by its prescribed weight. A component that loses its last exposed ends cannot interact with any future crossing, so it may be evaluated immediately. Grouping equal residual matchings is ordinary distributivity. These facts give the induction step. At the end, the only possible matching is empty and the stored coefficient is  $B(D)$ .  $\square$

We fix  $A = 2$  in  $\mathbb{F}_p$ . Both  $A$  and  $\delta = -17/4$  are nonzero, so every division in the normalization is valid. If the computed value is not one, the knot is nontrivial. Equality is inconclusive, regardless of whether it comes from an actual polynomial equality or a finite-field collision. There is no randomness and no estimated false-positive probability.

The default stage limits are 2,048 live matching states and 50,000 smoothing transitions. Reaching either limit skips the filter and preserves completeness of the overall algorithm. A global time-limit exception propagates instead. For the supplied Conway and Kinoshita–Terasaka diagrams, the filter uses at most five live states, with 62 and 68 transitions respectively. Their normalized field value is 267477248, not one. These are computations on the input PDs, not table lookups or recognition by filename.

### 5.3 Orientation and witness details

The old `signs()` convention records the incoming over-port but does not separately account for which under-port is incoming. Its Alexander matrix simultaneously treats slot zero as the incoming under-port, so those choices compensate. Replacing the sign formula alone would be a mistake.

For the Jones normalization we require a genuine oriented local sign. The new code traverses both strands at each crossing, records incoming slots  $u$  and  $o$ , and uses sign  $+1$  when  $(o - u) \bmod 4 = 3$ , and  $-1$  otherwise. Its symbolic and numeric Alexander matrices now also select the actual incoming/outgoing under-edges. A half-turn of one PD row shifts both incoming slots by two and therefore preserves the sign. The full polynomial is unchanged under the corresponding update. This is a consistency change, not a demonstrated correctness defect in the original Alexander recognizer.

The Jones result stores the prime, evaluation point, writhe, bracket, expected unknot bracket, crossing order, and a SHA-256 digest of the normalized PD. The `verify-jones` command replays the contraction and checks the decisive values. A full recognition output includes the reduced PD to which its witness applies; verification either matches that PD directly or reproduces the simplification. This is a *replayable calculation record*, not a claimed polynomial-time NP certificate. Verification may itself be expensive on a large frontier.

## 6 Visible connected sums without generator expansion

### 6.1 Finding and validating a two-edge split

Traverse the knot once and write down the crossing ID encountered at each visit. This cyclic Gauss sequence has length  $2n$ , with each crossing appearing twice. A proper cyclic interval is closed under crossing IDs if every ID that occurs in it occurs twice. The code searches for such intervals using one parity bit per crossing and a count of currently open IDs. An open count of zero marks a candidate; among candidates it prefers the most balanced split.

**Lemma 6.1** (Certified diagrammatic connected sum). *A proper closed Gauss interval determines a two-edge bond of the connected projection graph. Capping its two ends on either side gives a connected-sum decomposition of the represented knot.*

*Proof.* Each selected crossing has both visits inside the interval, so all its incident strand segments remain inside except the entry and exit edges. The interval's traversal connects all selected crossings, and the complementary interval connects all remaining crossings. Thus the two boundary edges form a minimal cut, or bond. In the spherical embedding a two-edge bond is dual to a separating curve crossing exactly those two edges. Lifting this curve to the usual separating sphere of the diagram produces a sphere meeting the knot twice. The two capped sides are its connected-sum factors.  $\square$

The implementation independently counts the cut edges, requires exactly two, identifies their labels on each side, and runs the ordinary single-component spherical PD validator again on both resulting diagrams. It retains the cut and parent PD as evidence. This is strictly a decomposition visible in the particular projection. A prime-looking projection of a composite knot may have no such interval; it is left intact, not classified as prime.

One split search costs  $O(n^2)$  parity updates. Recursing until no split is available gives a conservative  $O(n^3)$  combinatorial bound. Choosing balanced available cuts often does better, but no balanced cut is assumed to exist. Exact normalized factors are memoized within the calculation; nonidentical encodings of isomorphic factors may be recomputed.

## 6.2 Rank products and degree convolution

For two pointed diagrams joined at their marked arcs, the reduced cube complex is the tensor product of their reduced cube complexes over the coefficient field, up to the usual grading conventions. This can be seen directly from states: the two marked circles become one marked circle. Fix its label to  $x$ . All other labels are independent. A saddle involving the marked circle acts on this fixed label in exactly the same way before and after the join. A crossing in one factor changes only that factor's local state. Consequently the joined differential is  $d_1 \otimes 1 + 1 \otimes d_2$  over  $\mathbb{F}_2$ .

Over a field, every finite chain complex splits into its homology with zero differential and contractible two-term summands. Tensoring a contractible summand with any complex is contractible. Therefore

$$\widetilde{\text{Kh}}(K_1 \# K_2; \mathbb{F}_2) \cong \widetilde{\text{Kh}}(K_1; \mathbb{F}_2) \otimes_{\mathbb{F}_2} \widetilde{\text{Kh}}(K_2; \mathbb{F}_2). \quad (2)$$

In particular,  $r_2(K_1 \# K_2) = r_2(K_1)r_2(K_2)$ .

The code also preserves the available homological-degree information. If  $H_i(z) = \sum_h b_{i,h} z^h$  is the reduced cube-degree rank polynomial of a factor, then the result is

$$H(z) = \prod_i H_i(z), \quad r_2(K) = H(1), \quad \dim \text{Kh}^h(K; \mathbb{F}_2) = 2[z^h]H(z).$$

It divides each factor's unreduced dimensions by two, asserting that every coefficient is even, and performs ordinary integer coefficient convolution. No generators of the tensor product are instantiated. Coefficients have polynomially many bits in the total diagram size even when their numerical values are exponential.

An explicit user-supplied crossing order disables factorization. Otherwise returning factor orders instead of following that order would break the replay contract. Factored output marks `factorized=true`, supplies factor records, and uses an empty whole-diagram order. Its width counter is the maximum *factor* scan width, not a width measurement of a fictitious product scan.

## 6.3 An actual family-level asymptotic improvement

Trefoil sums establish the multiplicity obstruction, but their Alexander polynomial is already nontrivial, so they do not by themselves demonstrate an exponential improvement of the *default recognition pipeline*. For that purpose use the supplied eleven-crossing Conway PD, denoted  $C$ . Its independently checked data are

$$\Delta_C(t) = 1, \quad r_2(C) = 33,$$

and it has no decreasing R1/R2 move. Form the explicit connected-sum chain  $C^{\#k}$  by the constructor in `benchmarks/common.py`.

**Proposition 6.2** (Fixed-block family improvement). *On this explicit 11k-crossing family, the original unrestricted pipeline has an exponential lower bound from its final-generator representation, whereas the new pipeline has polynomial complexity in  $k$ .*

*Proof.* The joining edges connect distinct crossing vertices in different blocks, so they create neither a curl nor a new two-edge bigon. The remaining local crossing data are unchanged. Thus the original decreasing simplifier does not remove these blocks. The knots are nontrivial by (2), so a sound descending test cannot accept them. Multiplicativity of the Alexander polynomial gives  $\Delta_{C^{\#k}} = 1$ . The original recognizer therefore reaches its scanning backend. Its final minimal complex contains  $2 \cdot 33^k$  explicit objects, giving at least that many object constructions and retained objects.

For the new recognizer, decreasing and descending tests and both Alexander stages are polynomial. The bounded Jones stage either rejects the knot or aborts after its fixed transition ceiling, with polynomial work in the input size. In the latter case the visible factorization recovers fixed-size blocks. Their scans have size bounded independently of  $k$ , and the degree convolution has  $O(k)$  possible degrees and  $O(k)$ -bit coefficients. Split discovery and arithmetic are polynomial. This remains true without recognizing duplicate factors by memoization and without assuming anything about the Jones value.  $\square$

This is a restricted-family improvement, not an asymptotic claim about every prime or composite knot. A hidden connected sum can still require an exponential whole-diagram scan. The default Jones filter also happens to reject the measured three-summand example early, but the proposition does not rely on that lucky evaluation.

## 7 Complexity and resource semantics

### 7.1 What is, and is not, bounded

Let  $N_i$  be the number of objects in a partial complex,  $E_i$  its number of nonzero differential entries, and  $q_i$  the maximum circle-basis size used in that stage. Let  $P_i$  count Schur-complement update pairs actually attempted. A useful implementation-oriented description is

$$T = T_{\text{ordering}} + T_{\text{transfers}} + \sum_i O(P_i C(q_i)) + T_{\text{heap}},$$

where  $C(q_i)$  includes coefficient support iteration and integer-bitset costs. Compilation reduces repeated topological work, and pivot selection reduces  $P_i$  in favorable cases. Neither bounds  $N_i$  by a function of width alone.

A conservative general bound remains  $2^{O(n)}$ . Before simplification, there are  $2^n$  resolution states and at most exponentially many circle labelings. Every scanned summand comes from this finite expansion; cancellation does not increase the number of summands. Sparse matrices have at most quadratically many entries in that exponential dimension. A morphism has at most  $2^{O(n)}$  possible dot monomials, and its bitset operations cost at most exponential time. Multiplying these fixed numbers of exponential factors still gives  $2^{O(n)}$ . This is deliberately coarse and assumes normalized PD labels; reading unusually long external integer labels also costs their encoded length.

For the *uncapped scalar* Jones contraction alone, if the realized width is  $w$ , there are at most  $(w-1)!!$  matching keys at a cut. Per-key gluing takes polynomial work in  $w$ , giving a bound of the form  $\text{poly}(n, w) 2^{O(w \log w)}$  for that scalar stage. It does not transfer to the full categorified complex. The implemented greedy ordering has no proven worst-case small-width guarantee. No planar-separator ordering is claimed to have been implemented.

### 7.2 Practical ceilings

The recognition deadline begins before preprocessing. Callbacks occur in simplification, ordering, determinant elimination, topology compilation, object construction, and sparse cancellation. The budget is cooperative: one long Python or integer operation may run past the deadline before the next callback. The original 60-second trial actually returned its timeout after about 63.8 seconds; we use the conservative 60-second threshold in speedup lower bounds.

The object ceiling applies to generated objects in a crossing expansion. It is checked while their labels are being enumerated, before building all differential maps. A conclusive early obstruction may return

KNOTTED even when `max_objects=1`, because it never invokes that backend. Tests that intend to force backend exhaustion explicitly turn off the relevant filters.

An exhausted Jones-local budget means “skip this optional calculation,” not “the knot is hard” and not UNKNOWN. A global deadline or allocation failure yields UNKNOWN. With no global ceilings, the complete fallback is still executed after all inconclusive filters. Reported statistics are instrumentation rather than certified complexity bounds. In particular, `max_differential_entries` samples the pre-elimination matrix at each crossing; it does not observe every transient fill-in peak during elimination. The two implementations’ composition counters use slightly different accounting and should not be divided to claim an operation-count speedup.

## 8 Validation and independent checks

### 8.1 The test corpus

All 34 test methods pass. Nineteen originate in the supplied test suite; assertions about which filter terminates the pipeline and tests intended to exhaust the backend were adjusted for the new optional stages. The original test file is separately preserved under `baseline/tests/`. The changes do not silently redefine the expected ranks or invariant values.

The added checks cover 2,245 composition-basis pairs over all triples of noncrossing matchings through six exposed ends, comparing the compiled algebra with explicit set-valued surface composition. They test all 128 units of a three-arc endomorphism ring, transferred crossing maps on reachable partial states, and independent scalar bracket sums on 100 fresh diagrams. Forty PD mutations combine crossing-row half-turns, crossing permutations, and arbitrary edge relabeling; writhe, Alexander polynomial, and Jones normalization are required to remain consistent. Braid relations, canceling braid pairs, and positive and negative stabilizing curls are also tested.

For homology, the independent reduced dense-cube oracle enumerates states, labels the marked circle by  $x$ , constructs multiplication/comultiplication maps directly, and computes a binary matrix rank. It does not call the packed cobordism compiler. All 168 one-component closures among length-two and length-four words in the three-strand generators  $\{\pm 1, \pm 2\}$  agree with this oracle. One hundred additional random three-, four-, and five-strand knots with four to nine crossings are checked against it under both pivot policies. The first twenty of those also check  $d^2 = 0$  after every crossing. These are in addition to the baseline rank and family tests.

Connected-sum tests compare raw and factored homological-degree polynomials, not just totals. They include mixed trefoil/figure-eight sums, up to fifty trefoil summands, and two Conway summands. The three-Conway test checks Alexander triviality, lack of decreasing simplification, the default Jones verdict, and the factored complete fallback with Jones disabled. Input validation, incorrect crossing orders, witness tampering, replay of an unreduced witness, and global/local resource semantics are exercised as well.

### 8.2 The difficult 36-crossing result

The original benchmark supplied no completed rank for its difficult five-braid. The new raw backend returns unreduced rank 5898, reduced rank 2949. To avoid relying only on a fast rewrite agreeing with itself, the bundle includes `benchmarks/reference_markowitz.py`. It subclasses the original complex and changes only its cancellation scheduler. It keeps the original set-valued surface maps, original

crossing transfers, and original inverse routine. This calculation finishes in 7.30 seconds and matches the packed calculation in *every* unnormalized homological degree.

Separately, four new scans use the original 36-crossing PD and its 24-crossing R1/R2 simplification, each with greedy and natural crossing order. Every scan checks  $d^2 = 0$  at each stage and obtains reduced rank 2949. The two whole-PD orders also have identical degree distributions. Across simplification, total ranks are compared, not unshifted cube degrees. Full records are in `results/validation.json`.

The validation script additionally checks agreement of successful benchmark records in 45 workload/phase groups: ranks and degree distributions for rank runs, verdicts for recognition runs, and exact permutations for order runs. An incomplete original timeout does not count as a completed rank comparison. The separate original-algebra calculation supplies the missing hard-case comparison. These checks provide substantial independent evidence, but are not a machine-checked proof of every line of the implementation.

## 9 Same-host performance measurements

### 9.1 Protocol

The recorded environment is CPython 3.13.5, GCC 14.2.0, Linux x86-64 with glibc 2.41, on an AMD EPYC 9V74 host. The benchmark worker runs single-process Python code; it does not use the host’s multiple cores to parallelize an individual calculation. Each sample starts a new interpreter and new caches. Trials are sequential, with A/B order alternating between repetitions. The host was not reserved as an isolated benchmark machine, so the JSON retains the individual samples and ranges rather than implying stable microsecond precision.

The timer starts after imports, diagram conversion/validation, and a garbage collection, immediately before the requested operation. Crossing-order construction is included. “Recognition” therefore includes the recognizer’s preprocessing but excludes process startup and input parsing. Tiny cases can be dominated by startup in ordinary CLI use; the numbers below measure the algorithm, not a shell command’s total latency.

Most entries are medians of three samples. The unchanged original hard raw case, the original three-Conway timeout, and the auxiliary pivot ablations use one bounded sample each. A displayed  $> B$  means a trial exhausted its cooperative  $B$ -second budget; it is not a measured completion time. Ratios for those rows are lower bounds using  $B$  divided by the new median. A dash means that configuration was not run. The raw JSON has 252 trials, including all failures and auxiliary measurements.

### 9.2 Raw backend: no preprocessing and no factorization

The following comparison calls `khovanov_rank` directly on the original PD, disables connected-sum factorization in the new implementation, and uses the same greedy crossing order. It therefore measures actual backend acceleration rather than earlier rejection by a cheaper invariant. Ratio =  $T_{\text{original}}/T_{\text{new}}$  and  $r_2$  is reduced rank.

| Diagram             | $n$ | $r_2$ | Original (s) | New (s) | Ratio |
|---------------------|-----|-------|--------------|---------|-------|
| Random 3-braid (20) | 20  | 75    | 0.105        | 0.025   | 4.24  |
| Random 3-braid (40) | 40  | 321   | 0.533        | 0.096   | 5.53  |
| Random 4-braid (15) | 15  | 5     | 0.008        | 0.006   | 1.27  |
| Random 4-braid (21) | 21  | 25    | 0.272        | 0.072   | 3.75  |
| Random 4-braid (31) | 31  | 45    | 0.307        | 0.056   | 5.51  |
| Random 4-braid (41) | 41  | 109   | 0.856        | 0.167   | 5.12  |

| Diagram             | $n$ | $r_2$ | Original (s) | New (s) | Ratio  |
|---------------------|-----|-------|--------------|---------|--------|
| Random 5-braid (24) | 24  | 13    | 0.034        | 0.016   | 2.17   |
| Random 5-braid (36) | 36  | 2949  | > 60         | 0.897   | > 66.8 |
| Random 6-braid (31) | 31  | 85    | 0.402        | 0.083   | 4.83   |
| Unknot chain (32)   | 32  | 1     | 0.006        | 0.003   | 1.88   |
| Unknot chain (128)  | 128 | 1     | 0.061        | 0.012   | 5.05   |
| Unknot chain (256)  | 256 | 1     | 0.230        | 0.022   | 10.54  |
| $T(2, 11)$          | 11  | 11    | 0.007        | 0.004   | 1.72   |
| $T(2, 41)$          | 41  | 41    | 0.084        | 0.023   | 3.61   |
| $T(3, 5)$           | 10  | 7     | 0.010        | 0.008   | 1.22   |
| $T(3, 11)$          | 22  | 15    | 0.051        | 0.029   | 1.75   |
| Conway              | 11  | 33    | 0.035        | 0.010   | 3.36   |
| Kinoshita–Terasaka  | 11  | 33    | 0.036        | 0.011   | 3.18   |
| Hard unknot (8)     | 8   | 1     | 0.002        | 0.002   | 1.15   |

On the hard raw case the new sample range is approximately 0.895–0.933 seconds. The > 60-second original result gives a conservative improvement greater than 66-fold relative to the new median. No ratio uses the archive’s 600-second run on a different operating system or interpreter.

### 9.3 Complete recognition pipeline

These runs use each implementation’s default sequence of stages. They answer the decision problem rather than demanding an exact rank for every knot.

| Diagram                 | Original (ms) | New (ms) | Ratio    | New terminal stage |
|-------------------------|---------------|----------|----------|--------------------|
| Conway                  | 35.82         | 1.331    | 26.90    | Jones mod $p$      |
| Figure-eight            | 0.214         | 0.200    | 1.07     | Alexander mod $p$  |
| Determinant-one grid    | 0.815         | 0.354    | 2.30     | Alexander mod $p$  |
| Scrambled unknot grid   | 0.459         | 0.387    | 1.18     | R1/R2              |
| Hard unknot (8)         | 1.833         | 2.556    | 0.72     | Khovanov           |
| Kinoshita–Terasaka      | 38.05         | 1.299    | 29.28    | Jones mod $p$      |
| Five-braid (36)         | 8.731         | 2.280    | 3.83     | Alexander mod $p$  |
| Same knot, reduced (24) | 7.073         | 0.455    | 15.55    | Alexander mod $p$  |
| $T(3, 5)$               | 0.722         | 0.467    | 1.54     | Alexander mod $p$  |
| Trefoil                 | 0.194         | 0.182    | 1.07     | Alexander mod $p$  |
| Crossing-free unknot    | 0.045         | 0.053    | 0.86     | R1/R2              |
| Unknot braid (40)       | 6.668         | 6.982    | 0.95     | R1/R2              |
| Three Conway summands   | > 10000       | 4.311    | > 2319.7 | Jones mod $p$      |

The two Alexander-trivial named knots are rejected by an exact Jones obstruction. The three-Conway family amplifies that effect: the original pipeline exhausts ten seconds, while the new median is 4.31 milliseconds. This is a selected synthetic family, not a typical-case claim about all 33-crossing knots.

There is a real regression on the supplied eight-crossing hard unknot: approximately 1.83 ms becomes 2.56 ms. Its extra obstruction stages are inconclusive. On a workload dominated by similarly tiny hard unknots, disabling Jones and modular tests may be preferable; the raw backend itself is slightly faster in the recorded comparison. The new default favors avoiding expensive complete complexes on nontrivial knots rather than minimizing every tiny unknot’s overhead.

## 9.4 Rank products on visible connected sums

These entries request exact homology ranks. The original has no factorized backend; the new one uses decomposition and degree convolution. This is not merely multiplication of hard-coded known ranks: every distinct factor PD is scanned by the same exact implementation.

| Family              | $n$ | Reduced rank | Original (ms) | New (ms) | Ratio  |
|---------------------|-----|--------------|---------------|----------|--------|
| 2 trefoil summands  | 6   | $3^2$        | 2.376         | 1.348    | 1.76   |
| 4 trefoil summands  | 12  | $3^4$        | 15.02         | 1.553    | 9.67   |
| 6 trefoil summands  | 18  | $3^6$        | 136.82        | 2.384    | 57.39  |
| 8 trefoil summands  | 24  | $3^8$        | —             | 2.052    | —      |
| 10 trefoil summands | 30  | $3^{10}$     | —             | 2.447    | —      |
| 50 trefoil summands | 150 | $3^{50}$     | —             | 16.01    | —      |
| 2 conway summands   | 22  | $33^2$       | 1318.69       | 11.73    | 112.38 |
| 3 conway summands   | 33  | $33^3$       | —             | 11.98    | —      |
| 10 conway summands  | 110 | $33^{10}$    | —             | 18.34    | —      |
| 50 conway summands  | 550 | $33^{50}$    | —             | 162.67   | —      |

For fifty Conway factors the output rank has 76 decimal digits. The return value is a Python integer and a polynomial-size degree table, not a tensor basis of  $33^{50}$  elements. No original fifty-factor run was attempted or extrapolated into a purported completion time. The lower-bound argument in Section 6 explains the structural difference.

## 9.5 Ablations and crossing-order cost

The difficult case separates representation cost from pivot-order cost:

| 36-crossing raw backend         | Seconds | Reduced rank |
|---------------------------------|---------|--------------|
| Original algebra, original LIFO | > 60    | —            |
| Compiled algebra, LIFO          | > 10    | —            |
| Original algebra, new scheduler | 7.301   | 2949         |
| Compiled algebra, new scheduler | 0.897   | 2949         |

The original-algebra/new-scheduler row is both an ablation and an independent representation check. Its result is not used as the unchanged baseline for the headline speedup. The compiled/LIFO timeout shows that the packed algebra alone does not reproduce the default improvement on this input.

For order construction alone, the following family is the closure of  $\sigma_1\sigma_2\cdots\sigma_n$  on  $n+1$  strands. The two algorithms return identical orders; only their maintenance work differs.

| Crossings | Original (ms) | New (ms) | Ratio |
|-----------|---------------|----------|-------|
| 128       | 57.05         | 4.387    | 13.00 |
| 256       | 226.08        | 8.692    | 26.01 |
| 512       | 862.79        | 17.06    | 50.58 |

These measurements are consistent with the proved change from quadratic rescoring to heap-based  $O(n \log n)$  work per start. They are not measurements of a general polynomial-time unknot recognizer.

## 10 Using, auditing, and extending the implementation

### 10.1 Commands and API

The optimized package requires Python 3.10 or later and has no runtime third-party dependencies. From the extracted directory:

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot recognize examples/hard_unknot_8.json --check-d2
python -m fastunknot khovanov examples/random_braid5_36.json --no-factor
python -m fastunknot recognize examples/conway.json --output witness.json
python -m fastunknot verify-jones examples/conway.json witness.json
```

The Python entry points remain `Diagram`, `recognize`, `alexander_polynomial`, and `khovanov_rank`. The `recognize` call accepts individual switches for reduction, descending testing, Alexander, modular Alexander, Jones, and factorization. Both the CLI and API expose the pivot choice. CLI `-no-alexander` disables both numeric and full symbolic Alexander tests; `-no-modular` disables only the numeric stage. `-no-jones` and `-no-factor` are independent.

A complete positive or negative verdict exits zero. Invalid input or failed witness verification exits two; exhausted recognition budgets exit three. JSON explicitly reports `quasipolynomial_guarantee=false`. This is a useful machine-readable distinction from accepting a timing budget as a mathematical complexity guarantee.

### 10.2 Source organization

| Module or directory       | Responsibility                                                                                             |
|---------------------------|------------------------------------------------------------------------------------------------------------|
| <code>diagram.py</code>   | Input normalization, spherical/one-component validation, traversal, and orientation.                       |
| <code>geometry.py</code>  | Explicit reference/compatibility surface algebra and gluing structures copied from the original scanner.   |
| <code>algebra.py</code>   | Compiled component plans, local basis encoding, bitset composition, and crossing transfers.                |
| <code>scan.py</code>      | Partial objects, sparse differentials, cancellation, homological counts, and the rank entry point.         |
| <code>ordering.py</code>  | Versioned-heap greedy orders and explicit-order validation.                                                |
| <code>jones.py</code>     | Bounded scalar field contraction and witness replay.                                                       |
| <code>modular.py</code>   | Numeric Fox minors and exact modular obstructions.                                                         |
| <code>decompose.py</code> | Visible two-edge cuts, factor memoization, and degree-polynomial products.                                 |
| <code>alexander.py</code> | Retained exact symbolic polynomial arithmetic.                                                             |
| <code>recognize.py</code> | Stage orchestration and global resource semantics.                                                         |
| <code>tests/</code>       | Independent calculations and regression tests.                                                             |
| <code>baseline/</code>    | Unchanged original source and the original test file.                                                      |
| <code>benchmarks/</code>  | Input constructors, cold workers, runners, independent large-case validation, and report-table generation. |
| <code>results/</code>     | Raw trials, validation evidence, test output, and provenance.                                              |

The optimized hot path never calls the compatibility `compose` function. Retaining that explicit function

is intentional: existing callers can still inspect the original set-valued morphisms, and tests have a separately represented algebra to compare against. The package source is small enough to audit without requiring a specialized computer-algebra installation.

### 10.3 Reproduction and remaining technical work

```
python -m unittest discover -s tests -v
python benchmarks/run.py --profile full --output results/my-full.json
python benchmarks/validate.py --hard --output results/my-validation.json
python benchmarks/tables.py --input results/my-full.json
cd report
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The first command tests current source. The validation command compares the delivered benchmark records and reruns the four large debug scans. The full benchmark includes bounded original timeouts; the smoke profile is available for a quicker installation check. The result archive contains the compiled report, its TeX source and generated measurement macros, source code, raw evidence, and a patch relative to the original **fast/** package.

The most direct further engineering work is adaptive sparse-versus-dense morphism storage, a better treatment of unrevealed connected sums, and more structured cancellation priorities that account for coefficient support as well as graph degree. Those changes require new measurements: a more elaborate priority can cost more than the fill-in it saves. General tangle interfaces also require preserving the action of boundary algebra and derived tensor information; scalar rank multiplication at an arbitrary multi-ended cut would be unsound.

A genuine reconstruction of the quasi-polynomial hierarchy route needs its own executable compressed manifold and surface data structures, together with proved progress and bit-complexity invariants for the named topological operations. The present code supplies neither those operations nor a substitute proof. What it supplies is a materially faster exact recognizer, a verified resolution of the supplied raw timeout instance, and a precisely delimited family-level asymptotic improvement.

## A The hard input and its complete recorded rank distribution

The difficult input is the closure of the following five-strand braid, using the supplied package's signed-generator convention:

```
[-4,-3,4,4,-3,2,1,1,-1,-1,3,-4,-1,3,2,2,-2,-1,
-3,2,3,-3,-4,-4,-3,3,1,3,-2,-4,-4,3,-3,3,-4,-2]
```

The original-algebra validator and new packed engine obtain the same *unreduced* cube-degree polynomial:

$$\begin{aligned} P(z) = & 8z^{10} + 40z^{11} + 112z^{12} + 230z^{13} + 386z^{14} + 552z^{15} \\ & + 698z^{16} + 784z^{17} + 792z^{18} + 722z^{19} + 588z^{20} + 428z^{21} \\ & + 278z^{22} + 158z^{23} + 78z^{24} + 32z^{25} + 10z^{26} + 2z^{27}. \end{aligned}$$

Thus  $P(1) = 5898$  and  $r_2 = 2949$ . With the common greedy order, both successful representations have a maximum of 17,694 objects before crossing-stage elimination and 5,898 afterward; 16,884 pivot

cancellations are performed, and the maximal exposed boundary has eight ends. These object counts agree between the two representations, making it especially clear that the compiled algebra speedup is not caused by silently omitting a part of the complex.

The R1/R2-reduced 24-crossing diagram has Alexander coefficient list

$$[2, -19, 80, -206, 372, -512, 567, -512, 372, -206, 80, -19, 2].$$

Its determinant is 2949. Equality with the reduced Khovanov rank is a useful additional check here, not a rule used to infer the rank for arbitrary knots. The full raw and simplified PDs are shipped in `examples/`.

## B Provenance and interpretation of evidence

`results/provenance.json` records the SHA-256 of the uploaded `Knots.zip` and every original Python source copied into the baseline. Each baseline file was compared byte-for-byte with the extracted upload. `results/benchmarks.json` records the platform, every input constructor and word, every per-sample budget, outcome, elapsed time, and successful return value. The archived benchmark input file retains its original metadata for traceability, but the report generator reads only the new measurement file.

`results/test-output.txt` is the final successful 34-test run. `results/validation.json` stores all cross-record checks and the four large  $d^2$ -checked reruns. These records establish what was actually executed; none of the benchmark timeouts is represented as a completed calculation. `SHA256SUMS.txt` permits checking the delivered files after extraction. The software and accompanying new documentation use the supplied MIT No Attribution license. Referenced research papers retain their own licenses and are not included in this solution archive.

## References

- [1] M. Lackenby, *Unknot recognition in quasi-polynomial time*, supplied talk slides, 109 PDF pages. Author-hosted copy accessed 18 September 2026. <https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. In particular Sections 9–10 and Propositions 9.1 and 10.2. <https://arxiv.org/abs/2607.23350>.
- [3] D. Bar-Natan, *Fast Khovanov Homology Computations*, arXiv:math/0606318, 2006. <https://arxiv.org/abs/math/0606318>.
- [4] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, arXiv:1005.4346, 2010. <https://arxiv.org/abs/1005.4346>.
- [5] A. N. Shumakovitch, *Torsion of the Khovanov homology*, arXiv:math/0405474, revised version 2018. In particular Corollary 3.2.C. <https://arxiv.org/abs/math/0405474>.
- [6] M. Khovanov, *A categorification of the Jones polynomial*, Duke Mathematical Journal **101** (2000), 359–426; arXiv:math/9908171. <https://arxiv.org/abs/math/9908171>.
- [7] L. H. Kauffman, *Combinatorial Knot Theory and the Jones Polynomial*, arXiv:2204.12104, 2022. <https://arxiv.org/abs/2204.12104>.